

Class 6: R functions

Harshitha Palacharla (PID: A17775151)

Table of contents

Background	1
Write your first R function: <code>add()</code>	1
Write a <code>generate_dna()</code> function	4
Write a <code>generate_protein()</code> function	7
Generate random protein sequences of length 6 to 13	8
Are our peptides “unique in nature”?	9

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets)
- the *body* (the lines of R code that do the work of the)

Write your first R function: `add()`

Q1a: Your first version of the function should add two input numbers together.
For example, `add(4, 7)` should return 11.

Here we will create a function to add some numbers. Let’s call it `add()`

Input arguments can be either “**required**” or “**optional**”. The latter have fall-back default values that will be used if the user does not specify them.

```
add <- function(x, y = 0) {  
  x + y  
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

```
add(4 + 7)
```

```
[1] 11
```

Q1b: For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add(c(4,7,10))`.

```
add <- function(x, y=0) {  
  sum(x, y)  
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

Q1c: Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2.

```
add <- function(x, ...) {  
  sum(x, ...)  
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add( c(4,7,10) )
```

```
[1] 21
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the **return()** function call.

```
add <- function(x, y=0, z=0) {  
  sum(x, y, z)  
  cat("Is it break time yet?" )  
}
```

```
add(10, 100)
```

```
Is it break time yet?
```

```
add <- function(x, y=0, z=0) {  
  ans <- sum(x, y, z)  
  cat("Is it break time yet?" )  
  
  return(ans)  
}
```

```
add(10, 100)
```

```
Is it break time yet?
```

```
[1] 110
```

Write a `generate_dna()` function

A useful function here is the “base R” `sample()` function:

```
sample(1:5, size=60, replace = TRUE)
```

```
[1] 1 1 2 3 5 2 5 3 4 2 3 4 4 1 2 4 5 2 3 4 1 2 1 4 1 3 2 4 4 3 1 4 1 4 3 3 4 4  
[39] 2 1 5 2 4 1 2 4 1 3 2 5 1 5 4 2 2 2 1 4 3 2
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G” and “T”...

```
sample(c("A", "C", "G", "T"), size=10, replace = TRUE)
```

```
[1] "A" "G" "C" "G" "T" "C" "C" "A" "G" "G"
```

Q2a: Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”.

```
generate_dna <- function(len=10) {  
  sample(x = c("A", "C", "G", "T"), size=len, replace = TRUE)  
}
```

```
# generate dna (default len = 10)  
generate_dna()
```

```
[1] "A" "G" "A" "C" "T" "T" "G" "A" "G" "G"
```

```
# generate dna (where len=6)  
generate_dna(len=6)
```

```
[1] "T" "G" "T" "G" "A" "T"
```

Q2b: Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”.

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

```
paste(c("A", "C", "G"), collapse = "")
```

```
[1] "ACG"
```

```
generate_dna <- function(len=10, single.element=TRUE) {  
  ans <- sample(x = c("A", "C", "G", "T"), size=len, replace = TRUE)  
  cat("Hello...")  
  
  if(single.element) {  
    cat("is it me you are looking for...")  
    ans <- paste(ans, collapse = "")  
  }  
  return(ans)  
}
```

```
generate_dna()
```

```
Hello...is it me you are looking for...
```

```
[1] "CCTATTCTGG"
```

```
generate_dna(single.element = FALSE)
```

```
Hello...
```

```
[1] "C" "T" "A" "G" "G" "C" "G" "G" "C" "C"
```

```
generate_dna <- function(len=10, single.element=TRUE) {  
  ans <- sample(x = c("A", "C", "G", "T"), size=len, replace = TRUE)  
  #cat("Hello...")  
  
  if(single.element) {  
    #cat("is it me you are looking for...")  
    ans <- paste(ans, collapse = "")  
  }  
  return(ans)  
}
```

```
generate_dna()
```

```
[1] "CCGCGGCGAA"
```

```
generate_dna(single.element = FALSE)
```

```
[1] "C" "G" "G" "C" "C" "T" "G" "G" "A" "G"
```

Q2c: Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example: len9
CGAAGGCTG

```
cat("hello \n there")
```

```
hello
there
```

```
generate_dna <-  
## sample() function randomly samples from the 4 nucleotide characters,  
## for a specified size (i.e. sequence length, default is len = 10).  
## Repetition of characters is enabled by replace argument.  
function(len=10, single.element=TRUE) {  
  ans <- sample(x = c("A", "C", "G", "T"), size=len, replace = TRUE)  
  
  ## if() function specifies that for single.element = TRUE (the default),  
  ## paste() function is run to return a single character string.  
  ## For single.element = FALSE,  
  ## a multi-element vector of single character nucleotides is returned.  
  if(single.element) {  
    ans <- paste(ans, collapse = "")  
  }  
  
  ## Format as FASTA with an ID line.  
  cat(paste(">len", len, "\n", sep=""))  
  cat(ans)  
  cat("\n")  
  ##  
}
```

```
generate_dna()
```

```
>len10  
AAACCTGGGG
```

```
generate_dna(11)
```

```
>len11  
TGCGAGCCGCT
```

Write a generate_protein() function

Q3: Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG".

```
generate_protein <- function(len=9) {  
  ## aa is a vector storing all amino acid characters.  
  aa <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T")  
  
  ## sample() function randomly samples from the aa object,  
  ## for a specified size (i.e. sequence length, default set to len = 9).  
  ## Repetition of characters is enabled by replace argument.  
  ans <- sample(x = aa, size=len, replace = TRUE)  
  
  ## paste() function returns a single character string.  
  paste(ans, collapse = "")  
}
```

```
generate_protein(6)
```

```
[1] "DCYETM"
```

Generate random protein sequences of length 6 to 13

Q4: Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

```
for(l in 6:13)
  cat(l, "\n")
```

```
6
7
8
9
10
11
12
13
```

```
for(l in 6:13) {
  cat(">id.", l, "\n", sep="")
  cat(generate_protein(l), "\n")
}
```

```
>id.6
DYQSYG
>id.7
NKYIATV
>id.8
PPQGWEHH
>id.9
DVTKARIQV
>id.10
KKPKMQTTYC
>id.11
IRTNCKNPTMY
>id.12
IIPWALKAHDQF
>id.13
FKEFKDKVYYDNL
```

Are our peptides “unique in nature”?

Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
6	100	100	N
7	100	100	N
8	100	100	N
9	100	100	N
10	90	100	Y
11	73	100	Y
12	80	83	Y
13	75	92	Y

Q5a: At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)?

The randomly generated peptides start to look “unique in nature” at a sequence length of 10 amino acids.

Q5b: Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe.

20^L refers to the number of unique amino acid sequences possible for a peptide of length L , where a smaller L corresponds to a smaller sequence space.

For a short peptide, its sequence space (e.g. only 20^3 for a 3-residue peptide) is substantially *smaller* than the size of the known protein universe, making it statistically probable for all unique sequences to be found in the nr (which has ~550 million sequences). By contrast, a long peptide’s sequence space (e.g. 20^{13} for a 13-residue peptide) is substantially *larger* than the size of the known protein universe, such that all random combinations cannot be covered by the nr.

Q6: A key biological observation is that MHC class II molecules preferentially bind peptides that are a certain minimum length. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

Based on my Q5 data alone, it appears that a minimum length of 10 amino acids is required to generate a unique peptide sequence. Thus, I hypothesize the minimum peptide length that the MHC class II molecules would preferentially bind to is approximately 10 amino acids (note: the real number may be larger, as some classmates found nr database matches for their 10-residue random sequence).

The MHC class II molecules of the immune system present their bound peptides to CD4+ T helper cells, which attempt to identify the peptide as “self” or “foreign”. As discussed in 5b, a short peptide’s sequence space is much smaller than the known protein universe, and thus a given short sequence may be found in both “self” and “foreign” proteins. Presenting such peptides to a CD4+ T cell would result in frequent misclassification of peptides as “foreign” or “self”, yielding inappropriate activation of the immune system (i.e. when a “self” peptide is misclassified as “foreign”) OR failure to activate an immune response against a pathogen (i.e. a “foreign” peptide is misclassified as “self”). The longer a peptide sequence, the more probable it is to be unique to the self or non-self protein-of-origin and the less likely it is to be misclassified / trigger the wrong immune response.